



Automatización del balanceo dinámico de recursos en clústeres mediante Ansible

Rodrigo Rodríguez Galindo

Objetivo General

- Optimización de infraestructura mediante Ansible, Podman y Control Groups (cgroups). Implementando un sistema automatizado que reduzca los recursos de los contenedores en clústeres Linux.

Objetivos Específicos

- Detectar automáticamente el fin de la fase pesada de los cálculos de cada clúster.
- Integrar Ansible para reaccionar a eventos en tiempo real.
- Identificar el contenedor activo del investigador.
- Ajustar CPU y RAM mediante podman.
- Generar reportes de liberación de recursos.

Problemática Actual

Desperdicio de Recursos:

Los procesos de investigación computacional suelen tener fases de carga desigual (una fase de cálculo intensivo seguida de una fase ligera de guardado de datos).

Actualmente, los recursos (CPU/RAM) permanecen asignados al 100% hasta que el proceso termina por completo, bloqueando a otros nodos del sistema de clústers.

Etapas del Proyecto

Etapas	Actor	Función Principal
0	Nodo Central Ansible	Preparación de Nodo Central Ansible.
1	Nodo del Investigador	Crear archivo señal (Trigger).
2	Ansible	Monitorear el nodo Rocky Linux vía SSH.
3	Ansible	Obtener el ID del contenedor activo.
4	Ansible	Modificar cgroups para reducir RAM/CPU.
5	Ansible	Eliminar archivo señal del Nodo Central.

Etapa 0: Preparación de Nodo Central

1. Instalación y configuración del sistema operativo Rocky Linux 10.1

2. Instalación de herramientas/software en el nodo central:

- epel-release
- Fastfetch
- Ansible-core
- Python3
- Ssh
- Podman
- Ifconfig
- Nmap
- Cgroups
- Git
- Visual Studio Code

Etapas 0: Preparación de Nodo Central

3. Conectar vía SSH: Nodo central con 4 nodos de investigadores
4. Generación de llave SSH para cada nodo de investigadores
5. Cambiar las 4 “IP_nodo_Investigador” por un nombre corto
6. Crear carpeta “proyecto-ansible” en raíz ~/
7. Configurar ansible.cfg
8. Configurar terminal remota Visual Code vía SSH

Etapas 1: INICIO

Antes de que Ansible haga algo, el proceso del usuario debe avisar que cambió de fase.

El script o algoritmo matemático del investigador, (corre dentro de un contenedor de Podman), debe ejecutar una instrucción final al terminar sus cálculos intensivos.

Se crea un "archivo señal" en una ruta conocida. Funcionando como señal de inicio para la automatización.

Etapa 2: Detección

El nodo central (Rocky Linux) debe estar monitoreando los nodos del clúster para reaccionar a la señal.

Se utilizará un playbook (.yaml) de Ansible que emplee el módulo **wait_for**.

Es decir, ansible se conecta por SSH al nodo específico de Rocky Linux y queda en modo espera a que aparezca el archivo señal dada por el nodo del investigador.

Etapa 3: Selección

Una vez detectada la señal, Ansible ejecuta un comando (usando el módulo **command** o **shell**) para identificar el ID del contenedor de Podman exacto del investigador.

Se guarda este ID en una variable de Ansible (usando **register**) para usarla en el siguiente paso.

Etapa 4: Ejecución

Modificar el hardware asignado sin detener la ejecución.

Ansible envía la instrucción **podman update** al nodo del investigador.

Se modifican los límites de cgroups (Control Groups) del kernel de Linux para reducir la CPU y la RAM a la mitad.

El proceso del usuario entra en su Fase de proceso más ligera con menos recursos, liberando un porcentaje para que el administrador clúster lo asigne a otros nodos de investigadores.

Etapa 5: Limpieza

Ansible elimina el archivo señal para evitar ejecuciones duplicadas usando el módulo **ansible.builtin.file** con **state: absent**

Requisitos de Hardware (Nodo central)

- Procesador de arquitectura x86_64 con 8 Núcleos(i5/R5)
- 8GB RAM
- SSD 250GB
- Tarjeta de red con soporte para Fast Ethernet (100Mbps)

Con qué se cuenta hasta ahora?

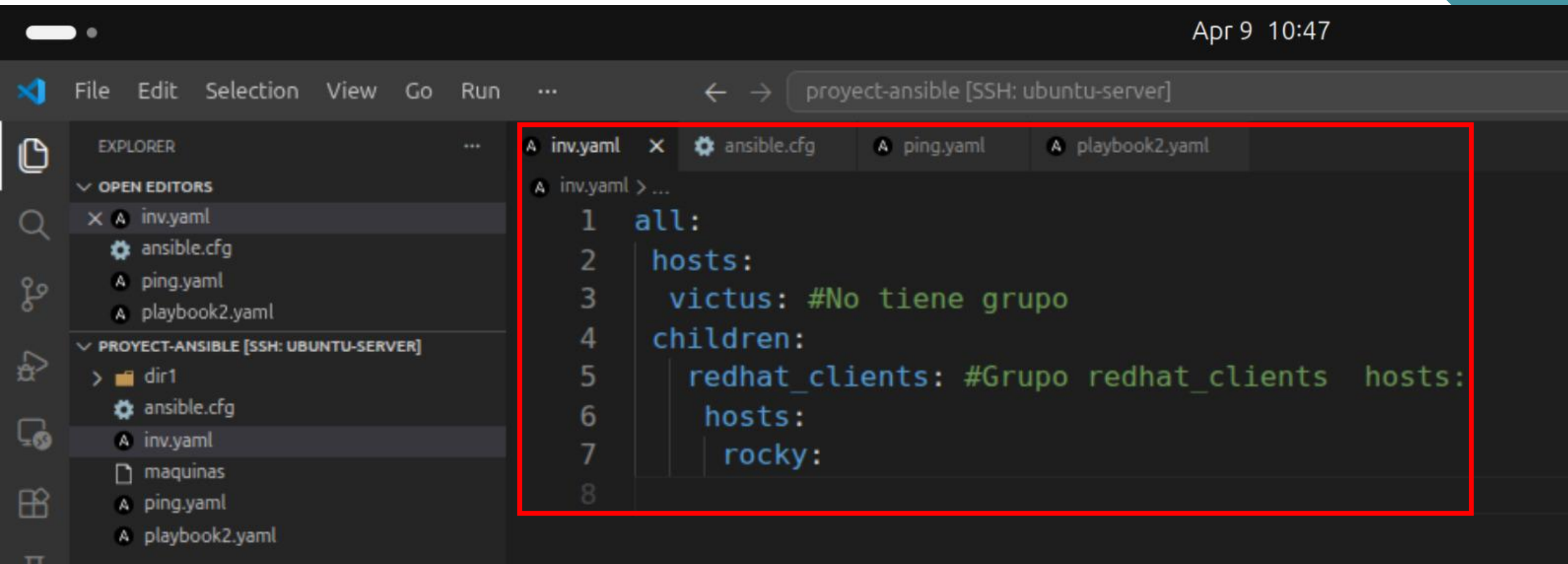
[illegible]

Pruebas de conexión SSH y ejecución de playbooks .yml entre **nodo Central (Ubuntu)** con **2 nodos clientes** (Rocky Linux y Linux Mint)

Conexión exitosa desde nodo central con 2 clientes ("victus" y "rocky")

```
rodrigo@rodrigo-HP-ProDesk-400-G6-SFF: ~/proyect-ansible
rodrigo@rodrigo-HP-ProDesk-400-G6-SFF:~/proyect-ansible$ ls
ansible.cfg  dir1  inv.yaml  maquinas  ping.yaml  playbook2.yaml
rodrigo@rodrigo-HP-ProDesk-400-G6-SFF:~/proyect-ansible$ ansible all -m ping
[WARNING]: Platform linux on host victus is using the discovered Python
interpreter at /usr/bin/python3.12, but future installation of another Python
interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
victus | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
rocky | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
rodrigo@rodrigo-HP-ProDesk-400-G6-SFF:~/proyect-ansible$
```

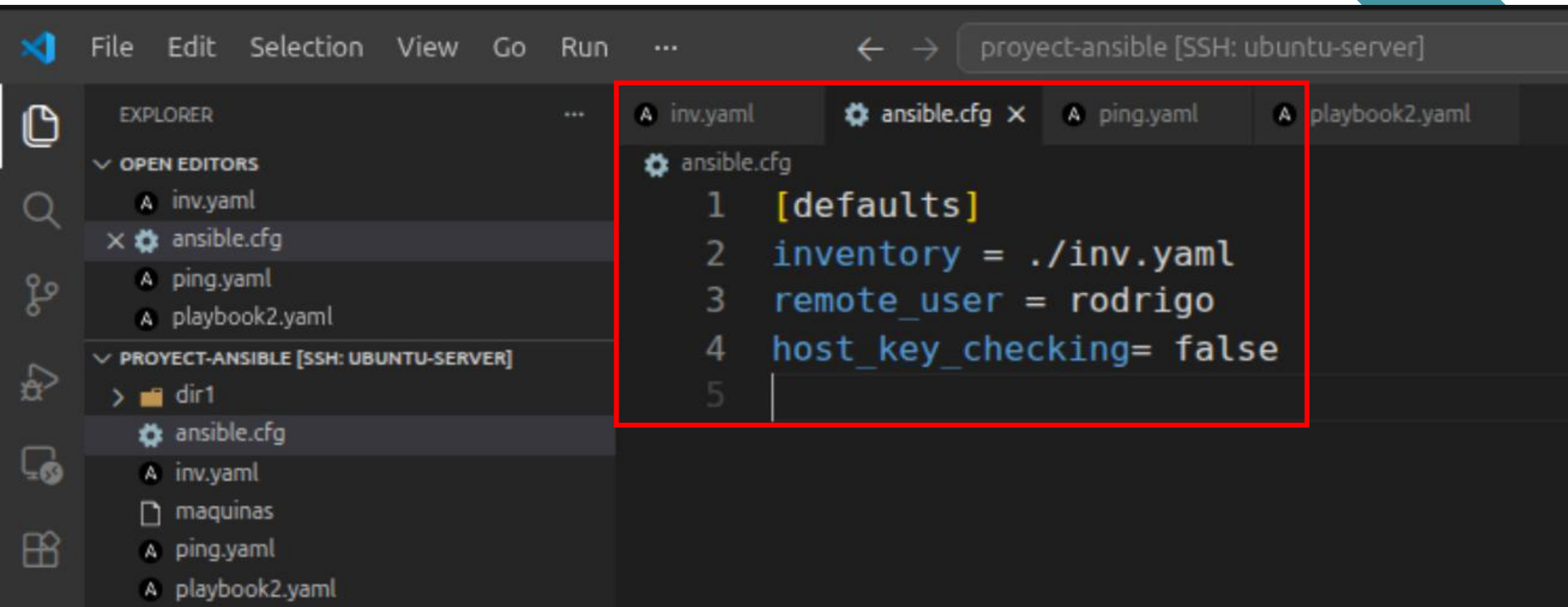

Prueba de configuración de inventario



The screenshot shows the Visual Studio Code editor interface. The top bar displays the date and time as 'Apr 9 10:47'. The menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', and a search icon. The Explorer sidebar on the left shows the project structure for 'PROJECT-ANSIBLE [SSH: UBUNTU-SERVER]', including a directory 'dir1' and files 'ansible.cfg', 'inv.yaml', 'maquinas', 'ping.yaml', and 'playbook2.yaml'. The main editor area has four tabs: 'inv.yaml', 'ansible.cfg', 'ping.yaml', and 'playbook2.yaml'. The 'inv.yaml' tab is active, showing the following YAML content:

```
1 all:
2   hosts:
3     victus: #No tiene grupo
4   children:
5     redhat_clients: #Grupo redhat_clients hosts:
6       hosts:
7         rocky:
```


Prueba de configuración de ansible



The image shows a screenshot of the Visual Studio Code editor interface. The top menu bar includes File, Edit, Selection, View, Go, Run, and a search icon. The Explorer sidebar on the left shows the project structure for 'proyect-ansible [SSH: ubuntu-server]'. It lists files: inv.yaml, ansible.cfg, ping.yaml, and playbook2.yaml. The 'ansible.cfg' file is highlighted in the Explorer. The main editor window displays the contents of 'ansible.cfg', which is highlighted with a red border. The configuration file contains the following text:

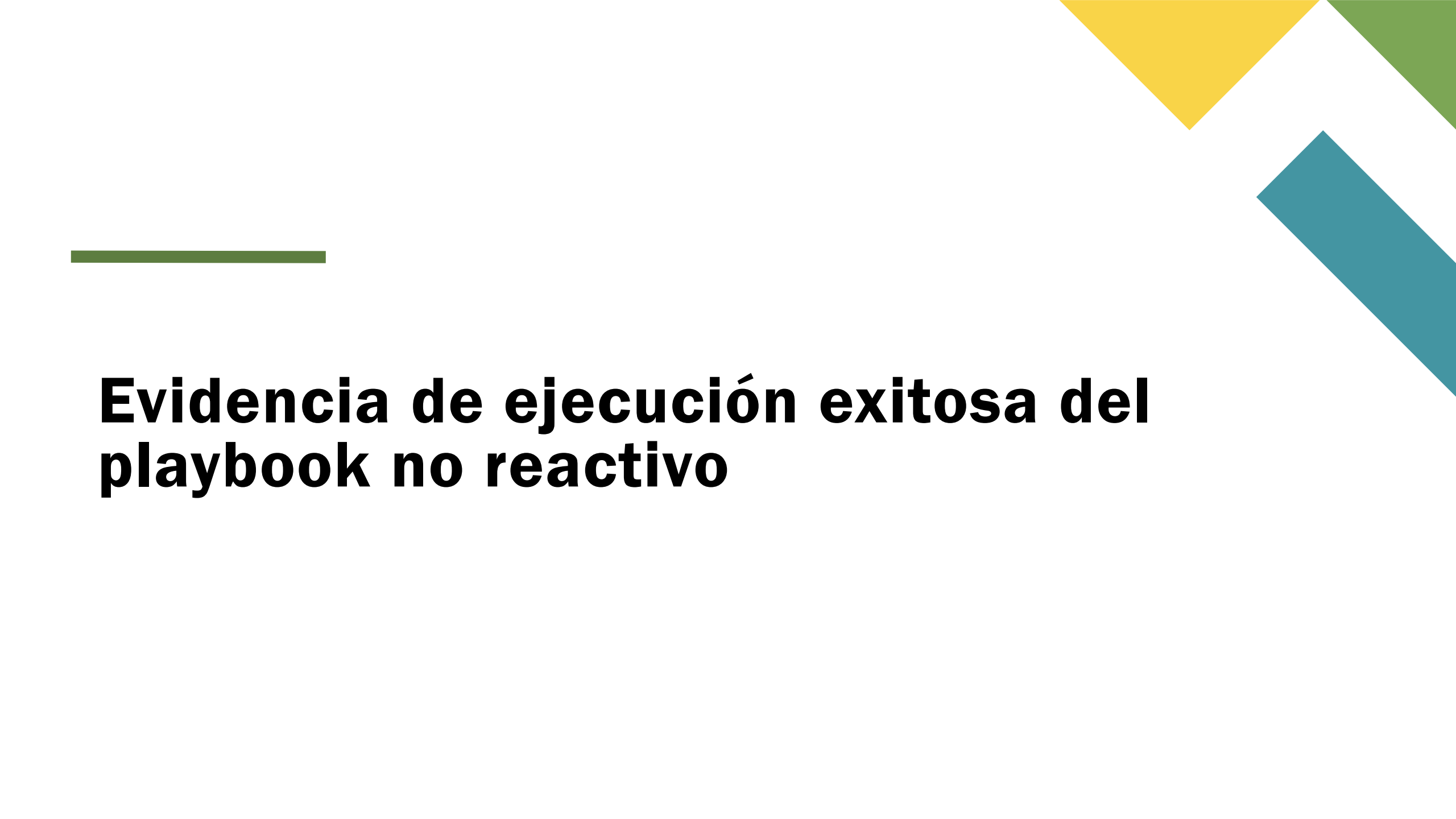
```
1 [defaults]
2 inventory = ./inv.yaml
3 remote_user = rodrigo
4 host_key_checking= false
5
```

Pruebas con Playbooks no reactivos

```
...  ← → project-ansible [SSH: ubuntu-server]

A inv.yaml  A ansible.cfg  A ping.yaml  A playbook2.yaml X

A playbook2.yaml > {} 1 > [] tasks > {} 0 > abc name
1  ---
2  - name: Primer play
3    hosts: all
4    tasks:
5      - name: Realizar un ping
6        ansible.builtin.ping:
7      - name: Crear archivo
8        ansible.builtin.command:
9          touch /home/rodrigo/archivo_creado
10
11 - name: Segundo play (instalar eh iniciar Apache)
12   hosts: victus
13   become: yes
14   tasks:
15     - name: Instalar servicio
16       ansible.builtin.apt:
17         name: apache2
18         state: present
19     - name: Iniciar servicio web
20       ansible.builtin.service:
21         name: apache2
22         state: reloaded
```



Evidencia de ejecución exitosa del playbook no reactivo

● **rodrigo@rodrigo-HP-ProDesk-400-G6-SFF:~/proyect-ansible\$** ansible-playbook -K playbook2.yaml
BECOME password:

PLAY [Primer play] *****

TASK [Gathering Facts] *****

ok: [rocky]

[WARNING]: Platform linux on host victus is using the discovered Python interpreter at /usr/bin/python3.12, but futu
could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpret

ok: [victus]

TASK [Realizar un ping] *****

ok: [victus]

ok: [rocky]

TASK [Crear archivo] *****

changed: [victus]

changed: [rocky]

PLAY [Segundo play (instalar eh iniciar Apache)] *****

TASK [Gathering Facts] *****

ok: [victus]

TASK [Instalar servicio] *****

ok: [victus]

TASK [Iniciar servicio web] *****

changed: [victus]

PLAY RECAP *****

rocky	: ok=3	changed=1	unreachable=0	failed=0	skipped=0	rescued=0	ignored=0
-------	--------	-----------	---------------	----------	-----------	-----------	-----------

victus	: ok=6	changed=2	unreachable=0	failed=0	skipped=0	rescued=0	ignored=0
--------	--------	-----------	---------------	----------	-----------	-----------	-----------

Gracias